W6: In-Gate Hit Rate via Deferred Writes (CALM-Editor)

Group 26 – CISC7021. Author: Vivi (Fong Chin Wai, MC653748). Scope: only the **in-gate parameter-write channel** of the W6 edit stream is measured here. Rev 2, 2026-11-07: adds the coverage-lift attempt (section 6) and the fluency/coverage frontier (section 7). Repo: github.com/FongChinWai-Vivi/CALM-Editor-CISC7021-Task1-Group26, previous HEAD 415527c.

1. Problem and root cause

The W4 closed-form writer inside the gate reached a 0.6% final-state hit, while the out-of-gate route (side memory / batched fallback) reached 78.8%. The bottleneck was not solver quality: the sequential gate applied one write per request, so edits that share a key space could never be solved jointly – every new write partially overwrote the previous one. The fix is structural: buffer the writes behind a window and re-solve the whole window cumulatively, then re-measure each committed member.

2. What changed in code

Modified: `easyeditor/editors/calm_editor.py`, `easyeditor/models/calm/calm_hparams.py`. New driver: `examples/w6_defer_stream.py`.

Change	What it does	Why
defer_window (W)	WRITE decisions are buffered for W edits, then committed in one solver call.	Lets edits that share a key be solved jointly instead of overwriting each other.
_flush_deferred	On flush the closed-form solution is recomputed over the accumulated constraints, then every member is re-measured with a real forward pass.	Removes the order dependence that produced the 0.6% floor.
_apply_with_guard	Per-flush NLL budget check (scale retry, then rollback to batched route).	Keeps fluency degradation bounded while the window grows.
Guard budget is spent per window (rev 2)	The counter is now cumulative inside a flush, and the straggler re-solve is guarded too.	Previously first solve + refine each got a full budget, so one window could drift twice the configured amount (measured +0.82 on a 0.5 budget).
defer_chunk (rev 2, default off)	Solves a window in contiguous sub-batches of size S.	Tested the hypothesis that one large solve underfits. It lost – see section 8.
New metrics	immediate_hit, survival@k buckets, final_state_hit (in-gate scope), final_state_hit_all_edits, param_coverage, param_hit_total.	The old single number mixed routing coverage with memory accuracy; every number now carries its own n / scope.
--data-offset	Slices records[offset : offset+edits] so a held-out split streams through the same code path.	Gives a clean records 100-199 replication without touching training-side data.

3. Headline results (n = 100 edits, 5 layers 4-8)

In-gate config unless stated: layers [4,5,6,7,8], l2 1, delta_budget 0.5, group_keys off, refine_passes 0. 'final' = final_state_hit over the in-gate parameter writes (the population the gate actually owns). 'final_all' / param_hit_total are diluted by the out-of-gate routes and must never be reported as memory accuracy.

Run	W	l2	layers	in-gate n	immediate	final (in-gate)	final_all	NLL delta	degen
A	1	10	4-8	90	0.222	0.000	0.000	+1.321	0.040
A	10	10	4-8	10	1.000	0.100	0.020	+1.023	0.010
A	25	10	4-8	6	1.000	0.167	0.010	+0.720	0.000
B	50	10	4-8	7	1.000	0.714	0.050	+0.373	0.010
B	100	10	4-8	7	1.000	1.000	0.070	+0.372	0.000
C	25	1	4-8	57	1.000	0.667	0.400	+0.945	0.010
C	50	1	4-8	61	1.000	0.918	0.560	+0.436	0.030
F	50	1	4-10	46	1.000	0.674	0.330	+1.255	0.140
F	100	1	4-10	47	1.000	1.000	0.470	+0.799	0.040
H (held-out, records 100-199)	50	1	4-8	23	1.000	0.870	0.220	+0.307	0.000

Reading: the 0.6% floor moves to 0.918 in-sample and 0.870 on held-out records, with NLL delta +0.44 / +0.31 and degenerate rate 0.03 / 0.00. The held-out population is only 23 in-gate writes, so the honest statement is '0.87 on 23 held-out writes, wide CI', never '92% memory accuracy'.

4. Survival and NLL

Run	survival 1-10	11-25	26-50	51+	NLL pre -> post
C (W=50, in-sample)	1.000 (n=3)	1.000 (n=7)	1.000 (n=24)	1.000 (n=32)	3.569 -> 4.005
H (W=50, held-out)	1.000 (n=3)	1.000 (n=7)	1.000 (n=24)	1.000 (n=32)	3.569 -> 3.876

No committed in-gate write is lost by later edits in the stream: the failure was a write-time overwrite problem, not long-horizon forgetting.

5. Reproducibility check after the rev-2 refactor

The baseline was re-run end to end after the guard refactor (run M, held-out records 100-199, W=50, refine 0, budget 0.5): in-gate n 23, final 0.8696, param_coverage 0.23, param_hit_total 0.20, NLL +0.3069, degen 0.0 – identical to H. The new code path therefore does not change the default behaviour.

6. Coverage-lift attempt (the open gap)

H leaves 75 of 100 held-out edits on the fallback route: coverage 0.23 is the weakest number in the W6 story, because a high in-gate hit rate over 23 writes says little about the stream. Three levers were tested.

Variant	in-gate n	param_coverage	final (in-gate)	param_hit_total	NLL delta	degen
H: W=50, refine 0, budget 0.5 (baseline)	23	0.23	0.870	0.20	+0.307	0.00
K: W=100, refine 1, budget per apply 0.5	59	0.59	0.932	0.55	+0.824	0.04
L: W=100, refine 1, budget per window 0.6	42	0.42	1.000	0.42	+0.673	0.01
L2: W=50, refine 1, budget per window 0.6	54	0.54	0.611	0.33	+1.007	0.11
I1: W=50, chunk 10 (dev slice 200-299)	38	0.38	0.132	0.05	+1.224	0.00
I2: W=50, chunk 10 + replay cache (dev slice)	15	0.15	0.667	0.10	+2.324	0.00

Result: coverage can be pushed from 0.23 to 0.59 on held-out records (K), and to 0.55 total hits, by (a) making the whole stream one window and (b) re-solving the stragglers once, guarded. That is the honest coverage win; the cost is fluency, see section 7.

Negative: chunked sub-batches (I1/I2) were a wrong hypothesis. Splitting a window does not improve conditioning; it multiplies guarded applies per window, lets the NLL drift accumulate (+1.22 / +2.32) while retention collapses (final 0.13 without replay, coverage 0.15 with replay, because replay constraints crowd out the new members). `defer_chunk` stays in the code as a documented dead end and defaults to 0.

7. The frontier: NLL cost per in-gate write

Run	in-gate writes	NLL delta	NLL per in-gate write
H (baseline)	23	+0.307	0.0133
L (guarded, per-window budget)	42	+0.673	0.0160
K (guarded, per-apply budget)	59	+0.824	0.0140

Every in-gate write costs roughly 0.014 NLL on gpt2-small, almost independently of the window design. Consequence: the acceptance pair 'coverage >= 0.5 AND NLL <= +0.5' is **not reachable** at this operating point – 0.5 coverage implies NLL +0.67 to +0.82. Two defensible targets instead: (i) NLL <= +0.5 with coverage ~0.42 (run L), or (ii) param_hit_total >= 0.5 with NLL <= +0.85 and degen <= 0.05 (run K). Recommendation to the team: state the NLL ceiling first, then report the coverage it buys, rather than asking for both.

8. Ablations that did not work

Variant	Result vs C (W=50)	Diagnosis
W=100 (single window, l2=1)	in-gate final 1.000 on n=7 only; final_all 0.07	Longer windows shrink the in-gate population the gate can afford (7 vs 61), so the high hit rate is over a much smaller set. Only interesting once the population is forced with refine (run K).
7 layers [4..10] (F, W=50)	final 0.674, NLL +1.255, degen 0.140	Extra layers add interference and dilute the per-layer displacement. Keep 4-8.
l2 = 10 (A/B rows)	final 0.714 (W=50), +0.373 NLL	Strong regularisation suppresses the update itself: fewer in-gate writes survive.
delta_budget 0.8 (D)	identical to C	The guard never fired at 0.5; the budget is not the binding constraint for the baseline.
group_keys ON + refine 3 (G)	final 0.012, NLL +7.059, degen 0.370	Fatal: with nearly all-distinct (subject, relation) pairs this degenerates to per-request solving and then re-applies the same delta. Both switches stay off.
defer_chunk 10 (I1/I2, section 6)	coverage 0.38 / 0.15, NLL +1.22 / +2.32	Chunking adds guarded applies per window and stacks drift; replay constraints crowd out new members. Dead end.

9. Pitfalls (do not repeat)

1) group_keys + refine together is destructive; it never converges and NLL explodes. 2) Do not report final_state_hit_all_edits as memory accuracy: it is diluted by the batched-route fallback (75 of 100 edits in H never reach the writer). 3) The in-gate

population changes with W and with refine, so every cross-run comparison must state n. 4) param_coverage must be logged beside param_hit_total, otherwise a high hit rate on a tiny population looks like a win. 5) The held-out run needs --data-offset; reusing the in-sample index silently reproduces the same records. 6) A guard that is applied per sub-solve instead of per window silently doubles the fluency budget; check the spent counter before quoting NLL.

10. Requests for the rest of the team

For Sonia (gate / side-memory interface): the gate interface must carry defer_window and the per-window delta budget through, and a WRITE decision has to be bufferable instead of committing on the spot. If the gate decides WRITE and then cannot defer, W>1 is unusable and the 0.6% behaviour returns. The WRITE/ROUTE/REJECT distribution must be logged together with the window size, otherwise the distributions are not comparable.

For Tang (thresholds / ablation alignment): the gate threshold was ROC-tuned under W=1 semantics and must be re-calibrated at W>1; the reachable in-gate population shifts from 90 to 23-61 writes depending on W and refine. Tune on two objectives together (param_hit_total and NLL delta) with degenerate rate as a hard ceiling (<= 0.05), and fix the NLL ceiling before quoting coverage (section 7).

11. Reproduction

```
In-sample baseline: python examples/w6_defer_stream.py --defer-window 50 --edits 100
--layers 4,5,6,7,8 --l2 1.0 --delta-budget 0.5 --data-offset 0
Held-out baseline: same with --data-offset 100
Coverage variant K: --defer-window 100 --refine-passes 1 --delta-budget 0.5 --data-offset
100
```

Results land in EasyEdit/w6run/<tag>/results_defer_W<W>.json; the summary dict holds every metric above with its own scope string.

12. Status

Committed in this revision: calm_editor.py, calm_hparams.py, examples/w6_defer_stream.py and this report. Defaults that matter for reviewers: **refine_passes = 0**, **group_keys = False**, **defer_chunk = 0**, **defer_window = 1** – the W6 behaviour is opt-in via CLI flags, so the default stream is still the sequential W=1 path. Run artefacts (EasyEdit/w6run, EasyEdit/w6out) are not committed. All numbers are a single seed per configuration; at least three seeds are needed before any of this goes into the final paper table.